In [6]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
```

In [7]:
```
df = pd.read_csv(r"C:\Users\praka\Dataset .csv")
```

In [8]:
```
df.head()
```

Out[8]:

	Restaurant ID	Restaurant Name	Country Code	City	Address	Locality	Locality Verbose
0	6317637	Le Petit Souffle	162	Makati City	Third Floor, Century City Mall, Kalayaan Avenu...	Century City Mall, Poblacion, Makati City	Century City Mall, Poblacion, Makati City, Mak...
1	6304287	Izakaya Kikufuji	162	Makati City	Little Tokyo, 2277 Chino Roces Avenue, Legaspi...	Little Tokyo, Legaspi Village, Makati City	Little Tokyo, Legaspi Village, Makati City, Ma...
2	6300002	Heat - Edsa Shangri-La	162	Mandaluyong City	Edsa Shangri-La, 1 Garden Way, Ortigas, Mandal...	Edsa Shangri-La, Ortigas, Mandaluyong City	Edsa Shangri-La, Ortigas, Mandaluyong City, Ma...
3	6318506	Ooma	162	Mandaluyong City	Third Floor, Mega Fashion Hall, SM Megamall, O...	SM Megamall, Ortigas, Mandaluyong City	SM Megamall, Ortigas, Mandaluyong City, Mandal...
4	6314302	Sambo Kojin	162	Mandaluyong City	Third Floor, Mega Atrium, SM Megamall, Ortigas...	SM Megamall, Ortigas, Mandaluyong City	SM Megamall, Ortigas, Mandaluyong City, Mandal...

5 rows × 21 columns

In [9]:
```
df.tail()
```

Out[9]:

	Restaurant ID	Restaurant Name	Country Code	City	Address	Locality
9546	5915730	Naml\ Gurme	208	××stanbul	Kemanke×× Karamustafa Pa××a Mahallesi, R\ht\m ...	Karak×_y
9547	5908749	Ceviz A××ac\	208	××stanbul	Ko××uyolu Mahallesi, Muhittin ××st×_nda×× Cadd...	Ko××uyolu
9548	5915807	Huqqa	208	××stanbul	Kuru×_e××me Mahallesi, Muallim Naci Caddesi, N...	Kuru×_e××me
9549	5916112	A×××k Kahve	208	××stanbul	Kuru×_e××me Mahallesi, Muallim Naci Caddesi, N...	Kuru×_e××me
9550	5927402	Walter's Coffee Roastery	208	××stanbul	Cafea××a Mahallesi, Bademalt\ Sokak, No 21/B, ...	Moda

5 rows × 21 columns

In [10]:
```
df.isnull().sum()
```

Out[10]:
```
Restaurant ID           0
Restaurant Name         0
Country Code            0
City                    0
Address                 0
Locality                0
Locality Verbose        0
Longitude               0
Latitude                0
Cuisines                9
Average Cost for two    0
Currency                0
Has Table booking       0
Has Online delivery     0
Is delivering now       0
Switch to order menu    0
Price range             0
Aggregate rating        0
Rating color            0
Rating text             0
Votes                   0
dtype: int64
```

In [11]:
```
df.describe()
```

Out[11]:

	Restaurant ID	Country Code	Longitude	Latitude	Average Cost for two	Price range	
count	9.551000e+03	9551.000000	9551.000000	9551.000000	9551.000000	9551.000000	9
mean	9.051128e+06	18.365616	64.126574	25.854381	1199.210763	1.804837	
std	8.791521e+06	56.750546	41.467058	11.007935	16121.183073	0.905609	
min	5.300000e+01	1.000000	-157.948486	-41.330428	0.000000	1.000000	
25%	3.019625e+05	1.000000	77.081343	28.478713	250.000000	1.000000	
50%	6.004089e+06	1.000000	77.191964	28.570469	400.000000	2.000000	
75%	1.835229e+07	1.000000	77.282006	28.642758	700.000000	2.000000	
max	1.850065e+07	216.000000	174.832089	55.976980	800000.000000	4.000000	

In [12]:
```
df.shape
```

Out[12]:
```
(9551, 21)
```

In [13]:
```
df['Has Table booking'] = df['Has Table booking'].map({'Yes':1, 'No':0})
df['Has Online delivery'] = df['Has Online delivery'].map({'Yes':1, 'No':0})
```

In [14]:
```
features = ['Votes', 'Average Cost for two', 'Price range',
            'Has Table booking', 'Has Online delivery']
```

```
X = df[features]
y = df['Aggregate rating']
```

In [15]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta
```

In [16]:
```
lr = LinearRegression()
lr.fit(X_train, y_train)

y_pred_lr = lr.predict(X_test)
```

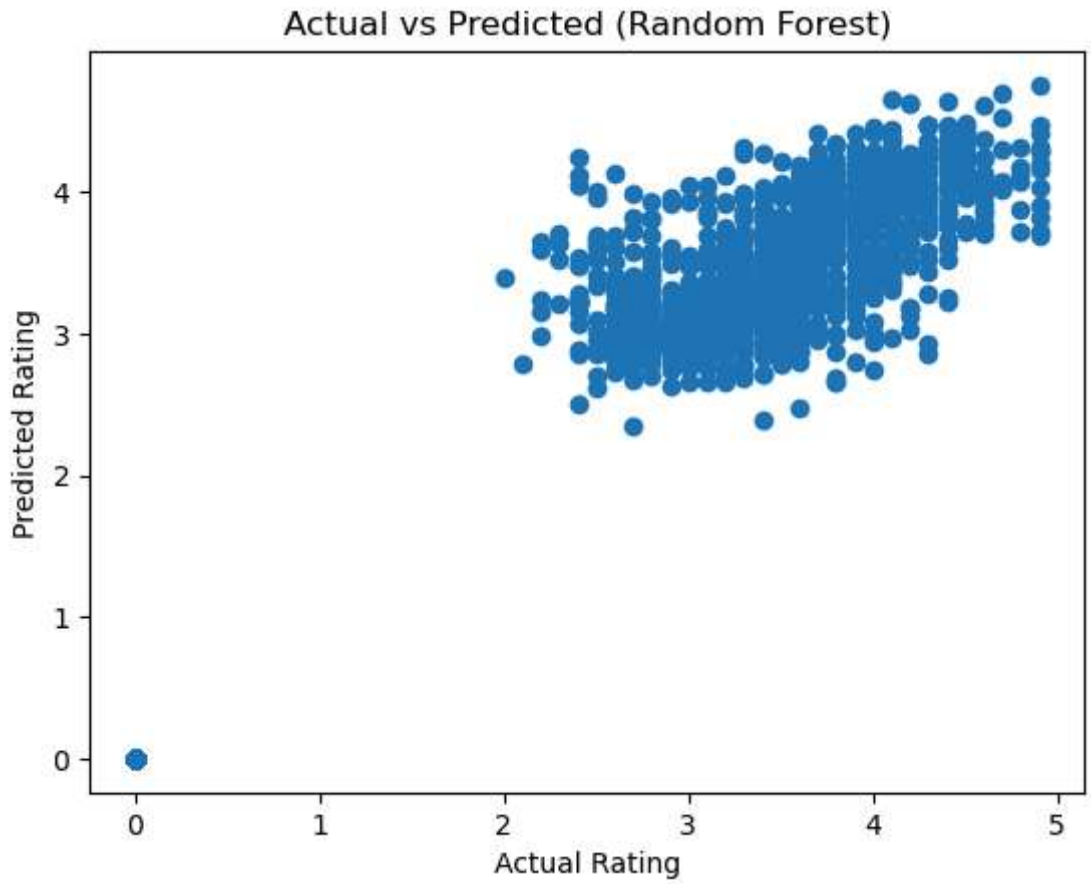
In [17]:
```
dt = DecisionTreeRegressor()
dt.fit(X_train, y_train)

y_pred_dt = dt.predict(X_test)
```

In [18]:
```
rf = RandomForestRegressor()
rf.fit(X_train, y_train)

y_pred_rf = rf.predict(X_test)
```

In [19]:
```
def evaluate(y_test, y_pred, model_name):
    print(f"{model_name}")
    print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
    print("R2 Score:", r2_score(y_test, y_pred))
    print("-"*30)

evaluate(y_test, y_pred_lr, "Linear Regression")
evaluate(y_test, y_pred_dt, "Decision Tree")
evaluate(y_test, y_pred_rf, "Random Forest")
```

```
Linear Regression
RMSE: 1.2947896642710524
R2 Score: 0.26344464090219477
------------------------------
Decision Tree
RMSE: 0.4538522077497544
R2 Score: 0.9095026523965715
------------------------------
Random Forest
RMSE: 0.3649740386773288
R2 Score: 0.9414764189783688
------------------------------
```

In [20]:
```
plt.scatter(y_test, y_pred_rf)
plt.xlabel("Actual Rating")
plt.ylabel("Predicted Rating")
plt.title("Actual vs Predicted (Random Forest)")
plt.show()
```


Actual vs Predicted (Random Forest)
Predicted Rating
Actual Rating
0
1
2
3
4
5

In [ ]: